Практикум 2.6 Графическое представление функции нескольких переменных

Упражнение 1. Создать функции, строящие графики функций $y = f(x)$, $y = g(x)$, на промежутке $[a;b]$ ($f(x) \le g(x)$ при $x \in [a;b]$) и показывающую вертикальной штриховкой область, заключенную между графиками. Входные аргументы – функции, границы отрезка $[a;b]$ и количество частей, на которые разбивается отрезок:

```
def plot_solid_fg(f, g, a, b, n):
    dx = (b - a) / n
    x = np.linspace(a, b)
    plt.plot(x, f(x))
    plt.plot(x, g(x))
    for i in range(n + 1):
        plt.plot(2 * [a + dx * i], [f(a + dx * i), g(a + dx * i)], 'g')

def plot_solid_f(f, g, a, b, n):
    dx = (b - a) / n
    x = np.linspace(a, b)
    plt.plot(x, f(x))
    plt.plot(x, g(x), '--')
    for i in range(n + 1):
        plt.plot(2 * [a + dx * i], [f(a + dx * i), g(a + dx * i)], 'g')

def plot_solid_g(f, g, a, b, n):
    dx = (b - a) / n
    x = np.linspace(a, b)
    plt.plot(x, f(x), '--')
    plt.plot(x, g(x))
    for i in range(n + 1):
        plt.plot(2 * [a + dx * i], [f(a + dx * i), g(a + dx * i)], 'g')

def plot_no_solid(f, g, a, b, n):
    dx = (b - a) / n
    x = np.linspace(a, b)
    plt.plot(x, f(x), '--')
    plt.plot(x, g(x), '--')
    for i in range(n + 1):
        plt.plot(2 * [a + dx * i], [f(a + dx * i), g(a + dx * i)], 'g')
```

Упражнение 2. Используя М-функции из упр. 1, построить области определения функции $f(x;y) = \sqrt{y+1-x^2} \cdot \sqrt{1+x^2-y}$.

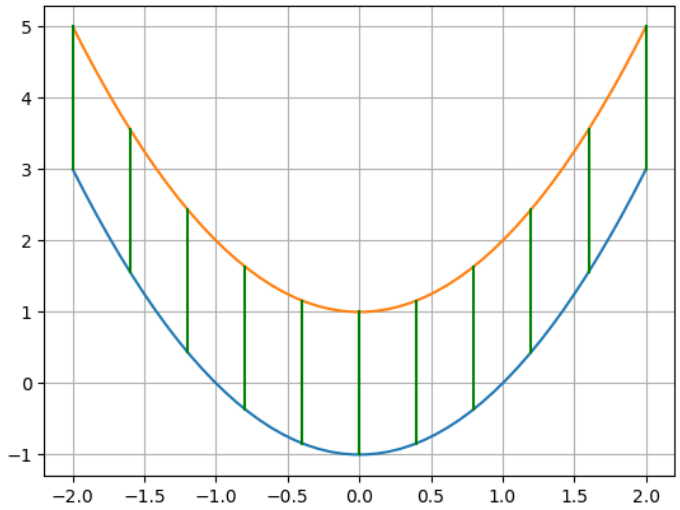
$y + 1 - x^2 >= 0$

$1 + x^2 - y >= 0$

$y >= x^2 - 1$

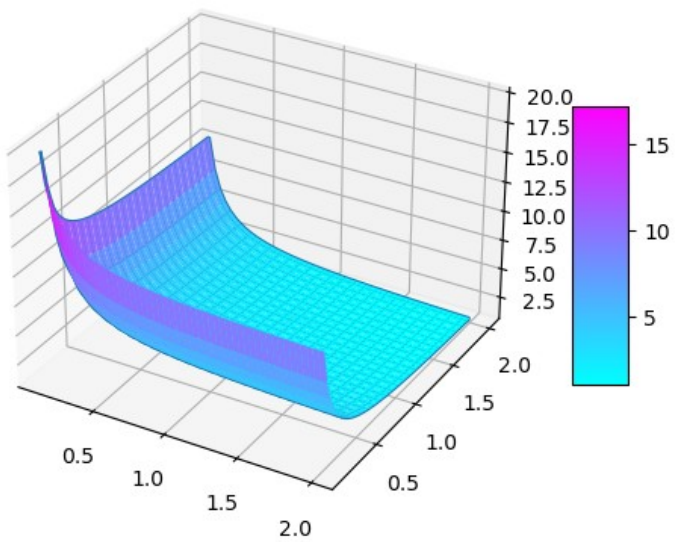
$y <= x^2 + 1$

```
plot_solid_fg(lambda x: x**2 - 1, lambda x: x**2 + 1, -2, 2, 10)
plt.grid()
```



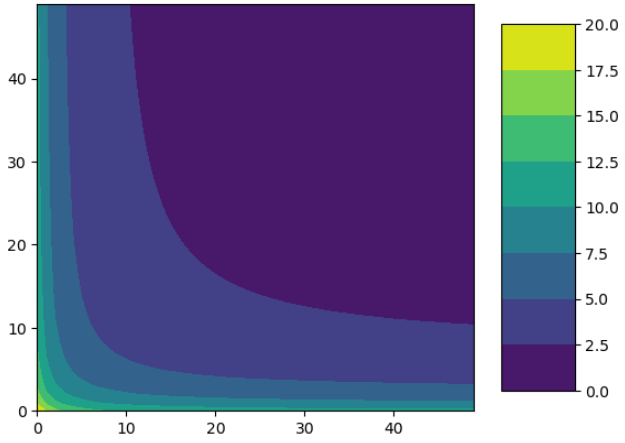
Упражнение 3. Построить каркасную поверхность, залитую цветом, с указанием соответствия цветов значениям для функции $z = 1/x + 1/y$, $x, y \in [0.1; 2]$.

```
fig, ax = plt.subplots(subplot_kw={"projection": "3d"})
xs = np.linspace(0.1, 2)
ys = np.linspace(0.1, 2)
X, Y = np.meshgrid(xs, ys)
Z = 1 / X + 1 / Y
ax.plot_wireframe(X, Y, Z, rstride = 2, cstride = 2)
surf = ax.plot_surface(X, Y, Z, cmap=cm.cool, linewidth=0)
fig.colorbar(surf, shrink=0.5, aspect=5)
plt.show()
```

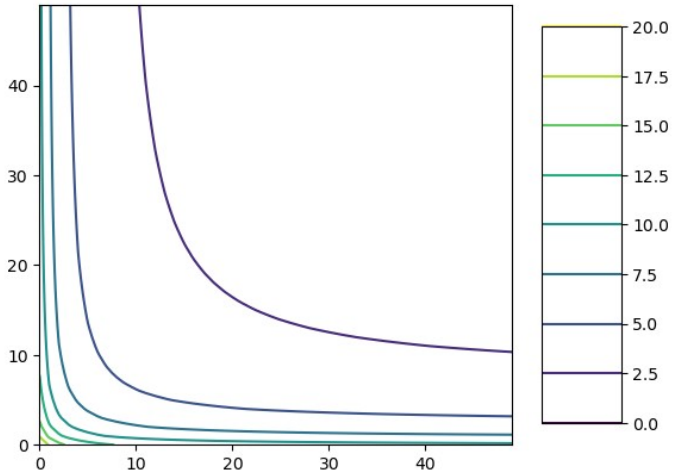


Упражнение 4. Для функции из упражнения 3 построить плоские линии уровня с заливкой и без неё; линии, поверхности, состоящие из линий уровня.

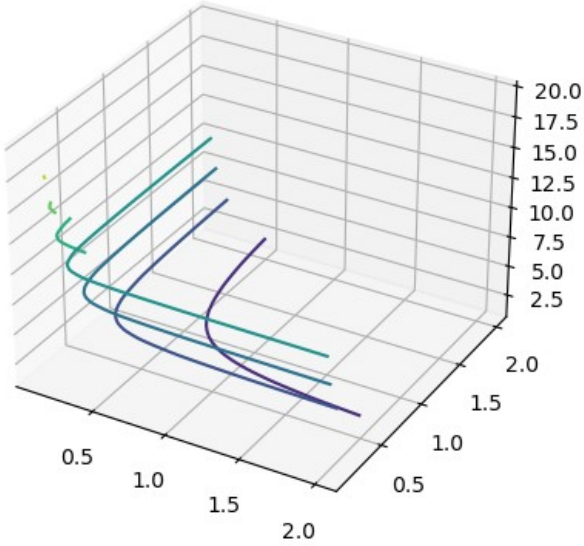
```
cs = plt.contourf(Z)
plt.colorbar(cs, shrink=0.9, aspect=5)
```




```
cs = plt.contour(Z)
plt.colorbar(cs, shrink=0.9, aspect=5)
```



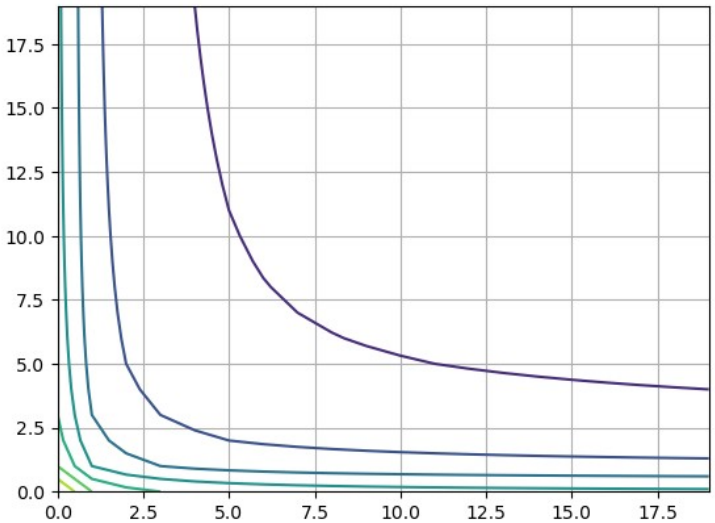

```
from matplotlib import cm
fig, ax = plt.subplots(subplot_kw={"projection": "3d"})
ax.contour(X, Y, Z)
```



Упражнение 5. Для функции из упр. 3 построить плоские линии уровня без нанесения и с нанесением значений функции с шагом 0.1. Нанести сетку.

```
xs = np.linspace(0.1, 2, num=20)
print(xs)
ys = np.linspace(0.1, 2, num=20)
X, Y = np.meshgrid(xs, ys)
Z = 1 / X + 1 / Y
cs = plt.contour(Z)
plt.grid()
plt.show()
```

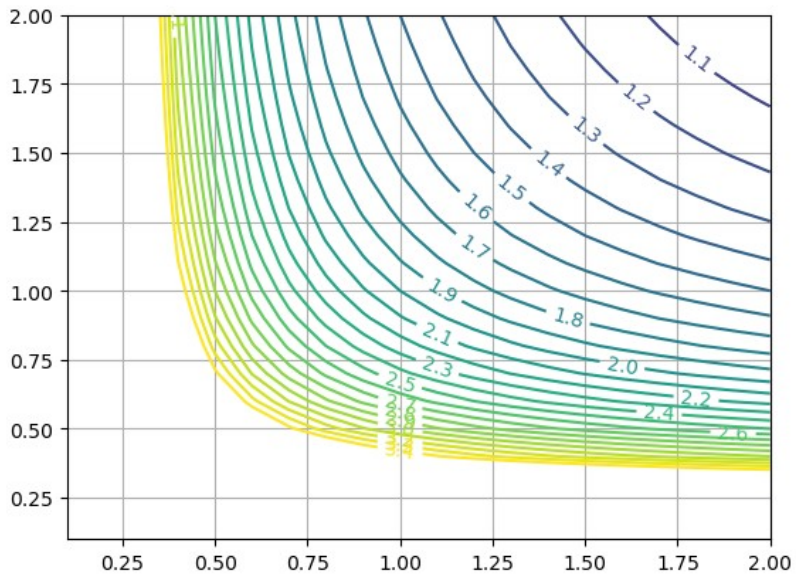
```
[0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.  1.1 1.2 1.3 1.4 1.5 1.6 1.7 1.8
 1.9 2. ]
```




```
xs = np.linspace(0.1, 2, num=20)
print(xs)
ys = np.linspace(0.1, 2, num=20)
```

```
X, Y = np.meshgrid(xs, ys)
Z = 1 / X + 1 / Y
levels = np.arange(0.5, 3.5, 0.1)
cs = plt.contour(X, Y, Z, levels=levels)
plt.clabel(cs)
plt.grid()
plt.show()
```

```
[0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.  1.1 1.2 1.3 1.4 1.5 1.6 1.7 1.8
 1.9 2. ]
```



Упражнение 6. Для функции из упр. 3 построить каркасную поверхность с с линиями уровня в разных проекциях вместе с самой поверхностью и отдельно от неё.

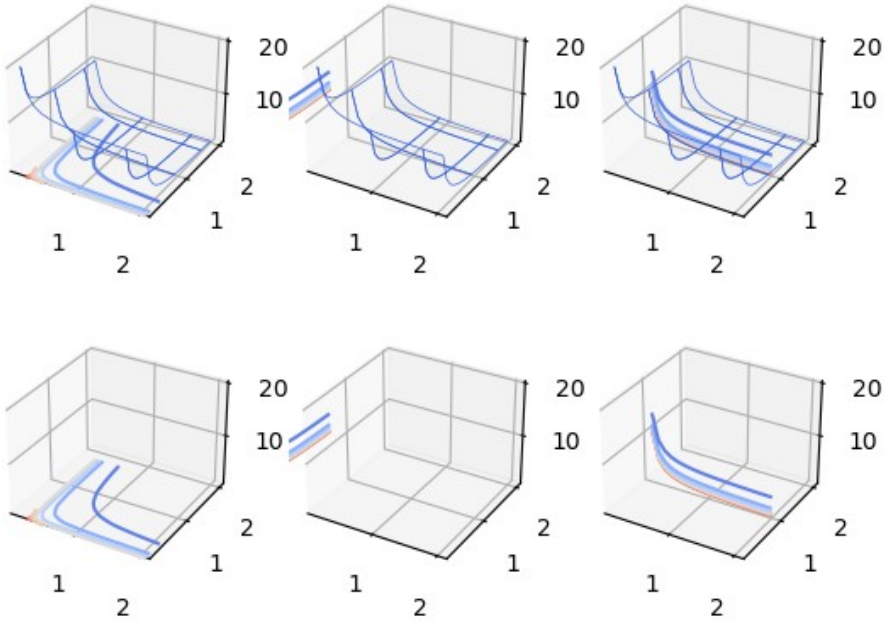
```
fig1 = plt.figure()
ax1 = fig1.add_subplot(231, projection='3d')
ax2 = fig1.add_subplot(232, projection='3d')
ax3 = fig1.add_subplot(233, projection='3d')
ax4 = fig1.add_subplot(234, projection='3d')
ax5 = fig1.add_subplot(235,projection='3d')
ax6 = fig1.add_subplot(236,projection='3d')

ax1.plot_surface(X, Y, Z, edgecolor='royalblue', lw=0.5, rstride=8, cstride=8, alpha=0.01)
ax2.plot_surface(X, Y, Z, edgecolor='royalblue', lw=0.5, rstride=8, cstride=8, alpha=0.01)
ax3.plot_surface(X, Y, Z, edgecolor='royalblue', lw=0.5, rstride=8, cstride=8, alpha=0.01)

ax1.contour(X, Y, Z, zdir='z', offset=-1, cmap='coolwarm')
ax2.contour(X, Y, Z, zdir='x', offset=-1, cmap='coolwarm')
ax3.contour(X, Y, Z, zdir='y', offset=1, cmap='coolwarm')

ax4.contour(X, Y, Z, zdir='z', offset=-1, cmap='coolwarm')
ax5.contour(X, Y, Z, zdir='x', offset=-1, cmap='coolwarm')
ax6.contour(X, Y, Z, zdir='y', offset=1, cmap='coolwarm')
```

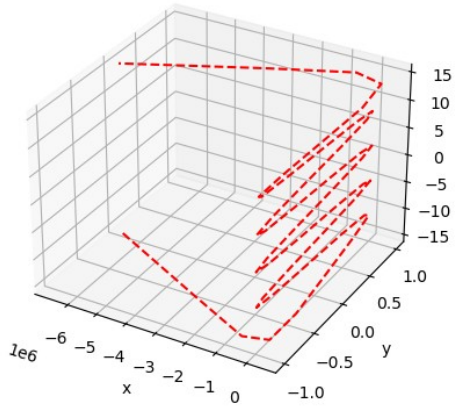
```
plt.show()
```



Упражнение 7. Построить пунктирную линию красного цвета, задаваемую уравнениями согласно варианту:

$$x = e^{|t|}\cos t, y = \sin t, z = t, t \in [-5\pi; 5\pi].$$

```
t = np.linspace(-5 * np.pi, 5 * np.pi)
x = np.exp(np.abs(t)) * np.cos(t)
y = np.sin(t)
z = t
fig1 = plt.figure()
ax = fig1.add_subplot(projection='3d')
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.plot(x, y, z, '--r')
```



Упражнение 8. Построить прозрачную каркасную поверхность, заданную параметрически, согласно варианту:

$$x = \sin u \cdot \cos v, y = 2\sin u \cdot \sin v, z = \cos u, u \in [0; 2\pi], v \in [0; 2\pi].$$

```
u = np.linspace(0, 2 * np.pi)
v = np.linspace(0, 2 * np.pi)
U, V = np.meshgrid(u, v)
XX = np.sin(U) * np.cos(V)
YY = 2 * np.sin(U) * np.sin(V)
ZZ = np.cos(U)
fig1 = plt.figure()
ax = fig1.add_subplot(projection='3d')
surf2 = ax.plot_wireframe(XX, YY, ZZ, rstride=3, cstride=3)
```

Упражнение 1С. Построить области определения следующих функций:

а) $f(x;y) = \ln(1 - x^2 - y^2) \cdot \ln y;$

б) $f(x;y) = \sqrt{\dfrac{4 - x^2 - y^2}{x^2 + y^2 - 1}}.$

a)

$$1 - x^2 - y^2 > 0$$

$$y > 0$$

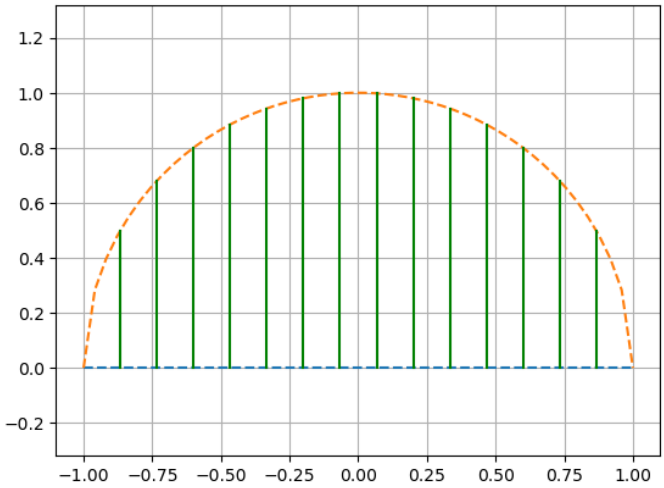
b)

$$(4 - x^2 - y^2)(x^2 + y^2 - 1) >= 0$$

$$x^2 + y^2 - 1 \neq 0$$

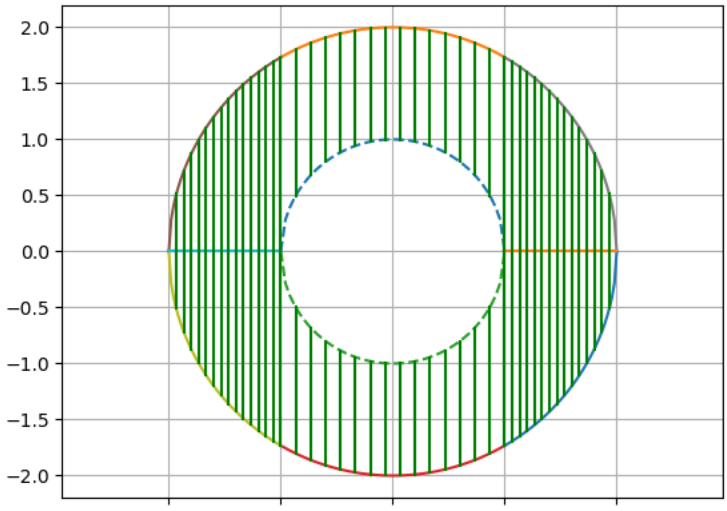
a)

```
plot_no_solid(lambda x: 0 * x, lambda x: (1 - x**2)**0.5, -1, 1, 15)
plt.grid()
plt.axis('equal')
```



б)

```
plot_solid_g(lambda x: (1 - x**2)**0.5, lambda x: (4 - x**2)**0.5, -1, 1, 15)
plot_solid_g(lambda x: -(1 - x**2)**0.5, lambda x: -(4 - x**2)**0.5, -1, 1, 15)
plot_solid_fg(lambda x: 0 * x, lambda x: (4 - x**2)**0.5, -2, -1, 15)
plot_solid_fg(lambda x: 0 * x, lambda x: (4 - x**2)**0.5, 1, 2, 15)
plot_solid_fg(lambda x: -(4 - x**2)**0.5, lambda x: 0 * x, -2, -1, 15)
plot_solid_fg(lambda x: -(4 - x**2)**0.5, lambda x: 0 * x, 1, 2, 15)
plt.axis('equal')
plt.grid()
```



Упражнение 2С. Построить каркасную поверхность, залитую цветом, с указанием соответствия цветов значениям для следующих функций:

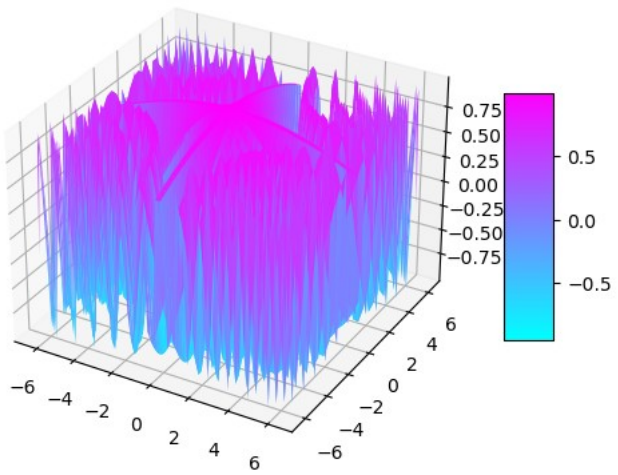
а) $z = \cos(xy)$, $x, y \in [-2\pi; 2\pi]$;

б) $z = \dfrac{1}{\sin x \cdot \sin y}$, $x, y \in [-2\pi; 2\pi]$ (шаг выбрать с учётом области определения).

а)

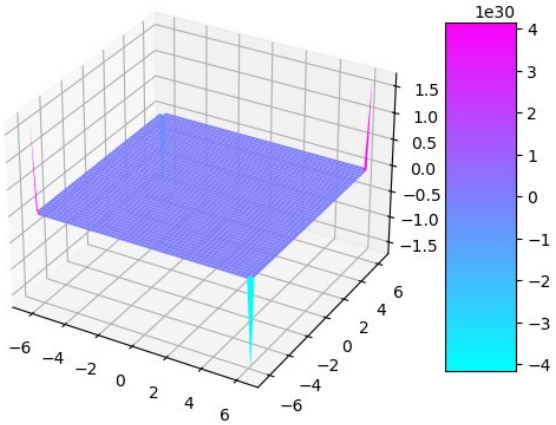
```
fig, ax = plt.subplots(subplot_kw={"projection": "3d"})
xs = np.linspace(-2 * np.pi, 2 * np.pi)
ys = np.linspace(-2 * np.pi, 2 * np.pi)
X, Y = np.meshgrid(xs, ys)
Z = np.cos(X * Y)

surf = ax.plot_surface(X, Y, Z, cmap=cm.cool, linewidth=0)
fig.colorbar(surf, shrink=0.5, aspect=5)
```



б)

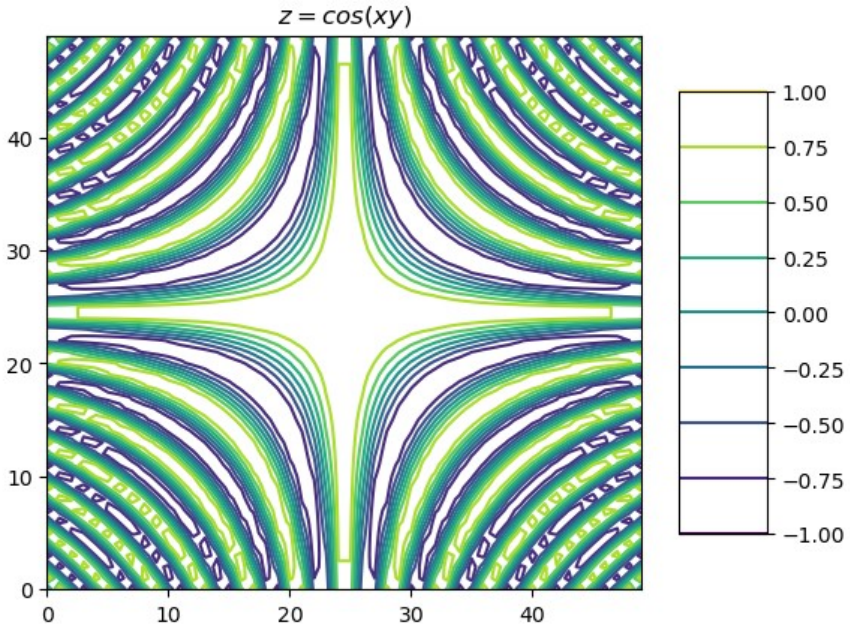
```
fig, ax = plt.subplots(subplot_kw={"projection": "3d"})
xs = np.linspace(-2 * np.pi, 2 * np.pi)
ys = np.linspace(-2 * np.pi, 2 * np.pi)
X, Y = np.meshgrid(xs, ys)
Z = 1 / (np.sin(X) * np.sin(Y))
surf = ax.plot_surface(X, Y, Z, cmap=cm.cool, linewidth=0)
fig.colorbar(surf, shrink=0.8, aspect=5)
```



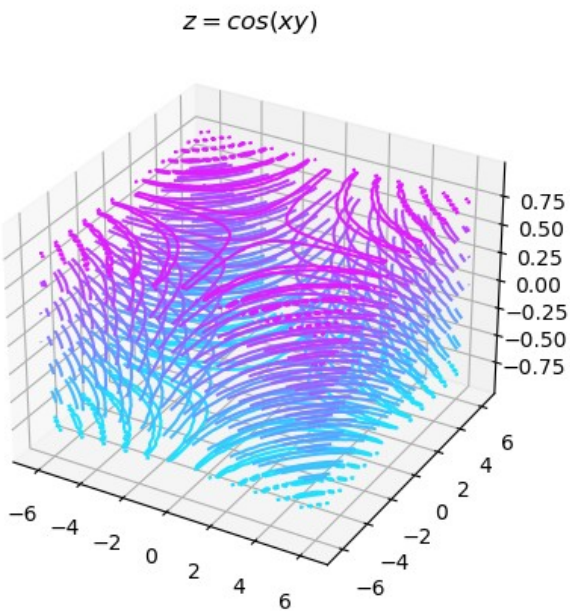
Упражнение 3С. Для функций из упр. 2С построить линии уровня и поверхности, состоящие из линий уровня.

а)

```
xs = np.linspace(-2 * np.pi, 2 * np.pi)
ys = np.linspace(-2 * np.pi, 2 * np.pi)
X, Y = np.meshgrid(xs, ys)
Z = np.cos(X * Y)
plt.title(r'$z=cos(xy)$')
cs = plt.contour(Z)
plt.colorbar(cs, shrink=0.8, aspect=5)
```

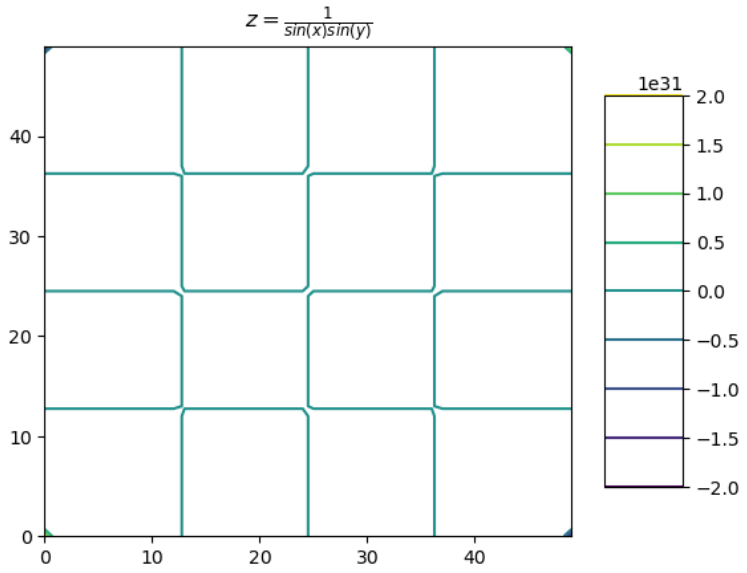



```
fig, ax = plt.subplots(subplot_kw={"projection": "3d"})
plt.title(r'$z=cos(xy)$')
ax.contour(X, Y, Z, cmap=cm.cool)
```

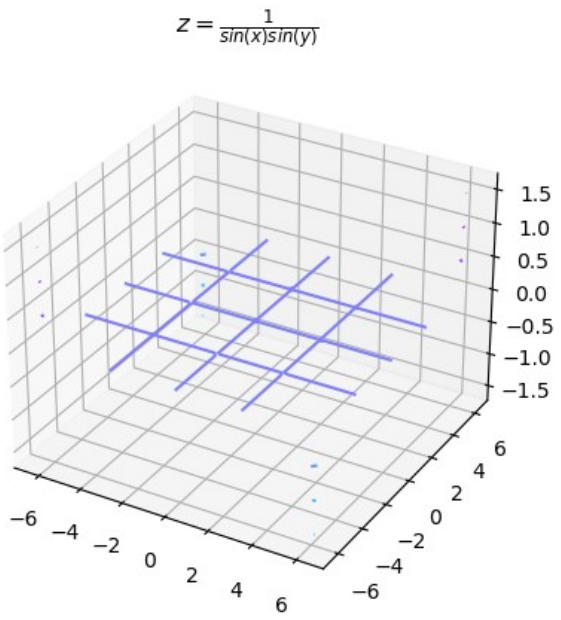


б)

```
xs = np.linspace(-2 * np.pi, 2 * np.pi)
ys = np.linspace(-2 * np.pi, 2 * np.pi)
X, Y = np.meshgrid(xs, ys)
Z = 1 / (np.sin(X) * np.sin(Y))
plt.title(r'$z=\frac{1}{sin(x)sin(y)}$')
cs = plt.contour(Z)
plt.colorbar(cs, shrink=0.8, aspect=5)
```




```
fig, ax = plt.subplots(subplot_kw={"projection": "3d"})
plt.title(r'$z=\frac{1}{sin(x)sin(y)}$')
ax.contour(X, Y, Z, cmap=cm.cool)
```

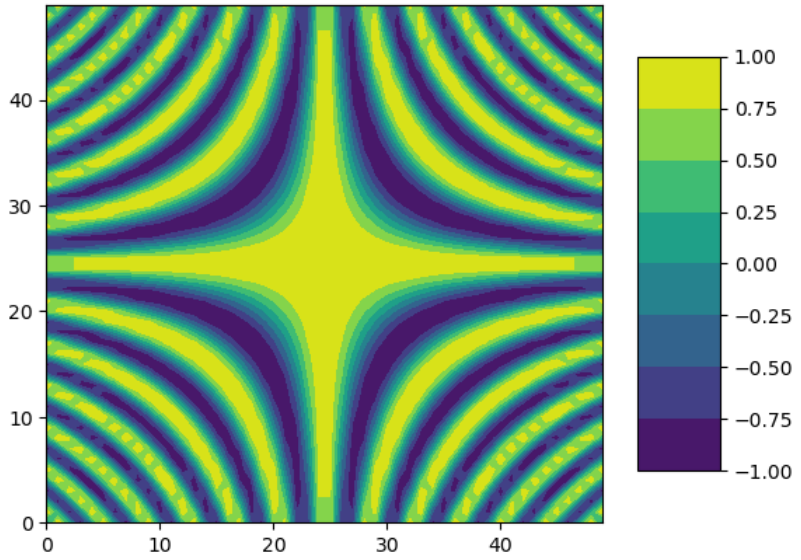


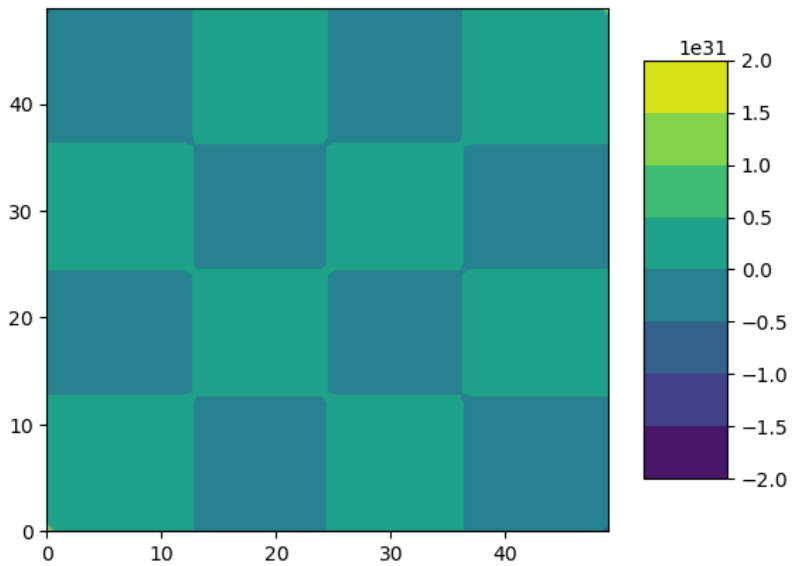
Упражнение 4С. Для функций из упр. 2С построить плоские линии уровня с заливкой промежутков между линиями уровня и шкалой соответствия цветов значениям функции.

a)

```
xs = np.linspace(-2 * np.pi, 2 * np.pi)
ys = np.linspace(-2 * np.pi, 2 * np.pi)
X, Y = np.meshgrid(xs, ys)
```

```
Z = np.cos(X * Y)
cs = plt.contourf(Z)
plt.colorbar(cs, shrink=0.8, aspect=5)
```



б)

```
xs = np.linspace(-2 * np.pi, 2 * np.pi)
ys = np.linspace(-2 * np.pi, 2 * np.pi)
X, Y = np.meshgrid(xs, ys)
Z = 1 / (np.sin(X) * np.sin(Y))
cs = plt.contourf(Z)
plt.colorbar(cs, shrink=0.8, aspect=5)
```